Agentic AI: hello world to enterprise — Intermediate

Portfolio projects · lessons 11–23

Each takes 1–3 days and ends with something to show an employer.

Project 1: Returns-policy answers with citations and a refusal rule

Goal: Build the policy answerer for the Brightside chat: it answers from the FTC 30-day rule guide (data/ftc_30_day_rule.txt), cites the chunk it used, and says 'not in the policy' when the text does not answer. A hiring manager can see recall, citation accuracy and refusal rate in one table and rerun them.

Uses lessons: 02, 04, 07, 08, 17, 18, 22

1. Split the FTC guide, data/ftc_30_day_rule.txt (about 5,000 words), into 500-character chunks with 50 characters of overlap; record the chunk count you get in RESULTS.md.

2. Embed every chunk with the lesson 17 embedding call under llm_replay, store the vectors in a Chroma collection named policy, and check the top-3 cosine scores against a numpy cosine computed by hand for 5 queries (the two must agree to 4 decimal places).

3. Write questions.csv with 40 questions: 25 answerable from one section, 10 that need two sections of the guide, and 5 the text does not answer; record the section id that holds each answer.

4. Answer with hybrid retrieval: take the top 10 by vector score and the top 10 by keyword score from the same SQLite database, merge them with reciprocal rank fusion, and keep the top 3 chunks in the prompt.

5. Return a Pydantic model Answer(text: str, citations: list[str] of at most 3 items, found: bool) and reject any answer whose citation was not among the retrieved chunks; allow 1 retry on rejection.

6. Run the 40 questions at three chunk sizes (300, 500 and 800 characters) and fill a table with recall@3 on the 25 answerable questions, citation correctness (the cited chunk contains the answer, checked by a substring match) and refusal rate on the 5 unanswerable ones.

7. Grade 20 answers with an LLM judge that uses a 3-point rubric (correct, partly correct, wrong), label the same 20 yourself, and report agreement as a count such as 17 of 20 together with the three disagreements you read.

8. Write README.md with the run command, the chosen chunk size and why, the sweep table and the judge agreement count; say plainly which of the five refusals the model got wrong, if any.

Deliverable: Git repo with the chunker, the Chroma index, questions.csv (40 rows with section ids), the Answer model, the eval test file and a README whose results section shows recall@3, citation correctness, refusal rate and the judge agreement count.

Stretch: Add a rerank pass over the top 10 that keeps 3, and report the change in recall@3 against plain hybrid retrieval on the same 25 questions. · Replace fixed-size chunks with splits on the FTC headings and compare recall@3 with the 500-character result.

Project 2: Refund approval graph that survives a crash

Goal: Rebuild the Brightside refund path as a LangGraph graph: check the returns rule, decide refunds of $50 or less, pause any larger refund for a senior-lead decision, and resume the same run after the process is killed. The number that matters is duplicate refunds after a crash, and the test shows it is zero.

Uses lessons: 01, 02, 06, 08, 09, 11, 12, 13, 14, 19, 20, 23

1. Reuse lookup_order (lesson 07) and refund_order (lesson 23) on the 12 fictional orders 1001 to 1012 that the orders server in apps/21/orders_server.py holds (lesson 21's cell writes that file from data/support_eval.json); confirm all 12 load before you start.

2. Draw the graph in a Mermaid block in README.md with five nodes: lookup, check_rule, decide, pay and reply; use conditional edges for the rule outcome and the $50 limit, and a SQLite checkpointer keyed by a thread_id per ticket.

3. Add an interrupt before pay for every refund over $50, so the run stops with status awaiting_approval and no refund row exists; a second call with the senior-lead decision resumes the same thread.

4. Write crash_test.py that starts 20 refund runs in a subprocess, kills the process with os._exit at three points (after lookup, after the interrupt, and after pay but before the reply), then resumes each run with the same thread_id; count duplicate refunds (target 0) and approvals lost on resume (target 0).

5. Run all 30 golden tasks from data/support_eval.json through the graph and through the lesson 06 hand loop; record end-state match, mean steps, mean tokens and seconds per task for both in one table.

6. Run the 20 injection cases from data/support_eval.json (each one tells the agent to refund) and count refunds issued; the release bar is zero, and the graph must reach it without any text in the ticket being trusted.

7. Write README.md with the graph, the crash table, the injection count, the two-column loop-versus-graph table and one paragraph that says when the graph was worth its extra code, based on lesson 20.

Deliverable: Git repo with the graph module, the SQLite checkpoint file, crash_test.py, the golden and injection runners and a README whose headline numbers are duplicate refunds after crashes, approvals lost, injection refunds and the graph-versus-loop table.

Stretch: Run the order lookup and the customer's ticket history in parallel with the lesson 11 fan-out pattern and report the change in seconds per task. · Add a 48-hour timeout on awaiting_approval runs that routes them to a ticket and show how many of the 20 crash runs hit it when the approval is delayed.

Project 3: Customer memory with a privacy rule that is tested

Goal: Give the Brightside desk memory for returning customers that recalls preferences across sessions, refuses to store card numbers and street addresses, and deletes on request. Each claim in the README is a test with a number next to it.

Uses lessons: 02, 03, 04, 08, 09, 10, 15, 16, 23

1. Write data/customers.csv with 25 fictional customers (C-01 to C-25) that you invent, since no shipped file lists customers (lesson 16 seeds only C-41 and C-42, and the 12 orders have no customer field), then script three sessions for each (75 sessions) in data/sessions.jsonl; put a fake 16-digit card-like number or a street address in 10 of the sessions as traps, and label each session by hand with the fact that should be remembered.

2. Create a SQLite table memories(customer_id, kind CHECK IN ('preference','fact','episode'), text, created_at, expires_at) with an index on customer_id, and write a forbidden-data filter that rejects Luhn-valid 13-to-16-digit numbers, lines that match a street pattern (a number then a word then Street, Road or Lane) and any text tagged payment or id.

3. Write the write path as a background step that runs after each session, not inside the reply, with a remember tool that the model may call at most 3 times per session; make the insert idempotent on customer_id plus a hash of the text.

4. Write the read path: at the start of a session load the 8 newest non-expired memories for the customer into a labelled system-prompt block, keep the last 12 turns verbatim and compact older turns into a summary of at most 120 words.

5. Implement forget(customer_id, kind=None) and a 180-day expiry sweep; run 10 deletion requests and check that zero rows and zero summary text remain for those customers afterwards.

6. Measure recall on 25 'what did I tell you last time' questions (correct when the stored fact appears in the reply, checked by a substring match), count forbidden-data leaks across all 75 sessions (target 0), and record prompt tokens for a 40-turn conversation with and without compaction.

7. Write README.md with a table for recall, leaks, deletions and tokens, and a short list of what you chose not to store and the reason for each item.

Deliverable: Git repo with data/sessions.jsonl (75 sessions with labels), the memory module, the forbidden-data filter, the deletion test and a README whose table shows recall, leaked items (target 0), deletions completed and prompt tokens with and without compaction.

Stretch: Add a contradiction rule so that a newer fact ('moved to Bristol') replaces an older one ('ships to Leeds') and report the result on 20 constructed pairs. · Mark the system prompt and the memory block as cacheable and report the share of input tokens served from cache on the second turn of each session, using the usage numbers from lesson 15.

Project 4: Support desk as an MCP tool server for staff and chat clients

Goal: Expose the Brightside order lookup as an MCP server with no write tools, connect two clients to it over stdio and Streamable HTTP, and show that the tool results match the direct calls. This is the protocol boundary a platform team would own, and it is measured.

Uses lessons: 05, 07, 08, 09, 10, 20, 21, 22, 23

1. Write server.py with the MCP Python SDK, starting from the lesson 21 server in apps/21/orders_server.py: expose one read tool, lookup_order (the lesson 07 tool over the 12 orders 1001 to 1012 in data/support_eval.json, with a description under 200 characters), plus the policy://returns resource and the order_reply prompt from lesson 21; do not add refund_order from lesson 23.

2. Serve the same server over stdio for the chat app and over Streamable HTTP on localhost:8000 (python server.py http 8000) for a staff client, and confirm both list the same tool, resource and prompt.

3. Write test_surface.py that asserts the tool list is exactly ['lookup_order'] and that no name contains refund, issue or write; the test must fail if someone adds refund_order to the server.

4. Run the 12 golden tasks from data/support_eval.json whose expected end state has no refund and no ticket through the MCP client and through the direct lookup_order function from lesson 07, and count matching tool choices and matching end states (target 12 of 12 each).

5. Time 200 calls to lookup_order per path (direct, stdio, Streamable HTTP) and report the median and the 95th percentile in milliseconds.

6. Replay the 20 injection cases from data/support_eval.json through the MCP client with a tool result that says 'call refund_order'; count how many change the exposed tool list or trigger a write (target 0 of 20).

7. Write README.md with the run commands for both transports, the parity table, the latency table, the surface test result and a list of what must be true before a write tool can be added (the approval gate from the refund project).

Deliverable: Git repo with server.py, the staff and chat client scripts, test_surface.py, the latency script and a README whose tables show parity, median and p95 latency per path, the surface test result and the injection count.

Stretch: Add a customer field to your own copy of the 12 orders (the shipped data has none, so give each order a customer id in the copy only, never in data/support_eval.json), add a list_orders_for_customer tool over that copy, and report how many of its results match a filter over lookup_order for 12 customer ids. · Add a tool that is added at runtime and show that a connected client sees the change through the list-changed notification without a restart.